Data Structures & Algorithms

GOLDEN GATE UNIVERSITY

Session 6 – Lab: Linked Lists and Stacks in Java

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

Lab goals

Build a Node class and a singly linked list in Java

Add insert, delete and search operations to the list

Extend the idea to a doubly linked list

Implement a stack on top of a linked list

Use a stack to reverse a string and to check brackets

Setup before you start

Item	What to do
Java	JDK installed; run java -version in a terminal
IDE	IntelliJ IDEA, Eclipse or VS Code
Project	New folder lab_lists_stacks with Node, SinglyLinkedList, DoublyLinkedList, LinkedStack, Main
Workflow	Write one method, run Main, compare with the expected output

Lab plan and timing

Part	Tasks	Minutes
Introduction and setup	Slides 1 to 4	9
1 Singly linked list	Tasks 1 to 3 and Checkpoint 1	21
2 Doubly linked list	Task 4	7
3 Stack on a linked list	Task 5	6
4 Stack applications	Tasks 6 and 7	9
Wrap-up	Extension, submission, recap	8

Part 1: the Node class (given)

```
public class Node {
    public int data;
    public Node next;

    public Node(int data) {
        this.data = data;
    }
}
```

Given code

Create Node.java exactly as shown

A new node has next = null by default

Public fields keep lab code short

Compile with javac Node.java

Task 1: insertFirst and display

```
public class SinglyLinkedList {
    private Node head;

    public boolean isEmpty() {
        return head == null;
    }

    public void insertFirst(int value) {
        // TODO 1: create node, link it, move head
    }

    public void display() {
        // TODO 2: walk from head, print each value
        // format: 8 -> 5 -> 3 -> null
    }
}
```

Your task

TODO 1: two lines of pointer work

TODO 2: a while loop using current

Print null at the end

Test with three insertFirst calls

Task 1 checkpoint

```
SinglyLinkedList list = new SinglyLinkedList();
list.insertFirst(3);
list.insertFirst(5);
list.insertFirst(8);
list.display();
// expected: 8 -> 5 -> 3 -> null
```

Common mistakes

head = n before n.next = head loses the list

Moving head while printing

NullPointerException: missing null check

Task 2: insertLast and deleteFirst

```
public void insertLast(int value) {
    Node n = new Node(value);
    if (isEmpty()) {
        head = n;
        return;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;   // find last
    }
    current.next = n;
}

public int deleteFirst() {
    // TODO 3: return the removed value;
    // throw an exception if the list is empty
}
```

Your task

Worked example: insertLast

The loop is needed: we keep no tail reference

TODO 3: write deleteFirst

Use throw new RuntimeException("List empty")

Task 3: find and delete

```
public boolean find(int key) {
    // TODO 4: true if key is in the list
}

public boolean delete(int key) {
    // TODO 5: remove the first node holding key
    // Cases:
    // 1 empty list    -> return false
    // 2 key in head   -> head = head.next
    // 3 key elsewhere -> stop at node BEFORE it
    //                    prev.next = prev.next.next
    // 4 not found     -> return false
}
```

Your task

Draw the list on paper first

Reuse the traversal loop

Test: delete first, middle, last, missing

Debugging pointer code

Draw

Boxes and arrows

Sketch the list before and after each operation.

Null

Check it

Before using x.next, ask: can x be null?

Step

One change at a time

Print the list after every operation; use the debugger.

Checkpoint 1: test your list

```
list.insertFirst(5);
list.insertLast(9);
list.insertFirst(2);
list.display();       // 2 -> 5 -> 9 -> null
list.delete(5);
list.display();       // 2 -> 9 -> null
list.deleteFirst();
list.display();       // 9 -> null
System.out.println(list.find(9));  // true
```

Checkpoint

Output must match the comments

Stuck for 5 minutes? Raise your hand

Finished early: add size() by counting while walking

Part 2: doubly linked list

```
public class DNode {
    int data;
    DNode prev, next;
    DNode(int data) { this.data = data; }
}

public class DoublyLinkedList {
    private DNode head, tail;

    public void insertFirst(int value) {
        DNode n = new DNode(value);
        n.next = head;
        if (head != null) head.prev = n;
        else tail = n;
        head = n;
    }
}
```

Given code

DNode adds a prev reference

insertFirst fixes 3 links

Empty list: head and tail both become n

Trace it: list 7, then insertFirst(4)

Task 4: work from both ends

Your task

insertLast(int value): mirror of insertFirst

deleteLast(): tail = tail.prev; tail.next = null

displayBackward(): start at tail, follow prev

Hints

Single-node list: head == tail

Empty list: throw an exception

Test: insertLast 1, 2, 3; backward prints 3, 2, 1

Part 3: stack on a linked list

```
public class LinkedStack {
    private Node head;      // head = top

    public void push(int x) {
        // TODO 6: same as insertFirst
    }
    public int pop() {
        // TODO 7: same as deleteFirst;
        // throw RuntimeException if empty
    }
    public int peek() {
        // TODO 8: return top, do not remove
    }
    public boolean isEmpty() { return head == null; }
}
```

Task 5

Reuse your Task 1 and 2 logic

Top of stack = head of list

pop and peek throw when empty

Task 5 checkpoint

```
LinkedStack st = new LinkedStack();
st.push(10);
st.push(20);
st.push(30);
System.out.println(st.pop());    // 30
System.out.println(st.peek());   // 20
System.out.println(st.pop());    // 20
System.out.println(st.pop());    // 10
System.out.println(st.isEmpty());// true
st.pop();   // must throw: Stack empty
```

Expected

Output: 30, 20, 20, 10, true

The last pop must throw an exception, not a NullPointerException

LIFO: last pushed, first out

Task 6: reverse a string

```
import java.util.ArrayDeque;
import java.util.Deque;

public static String reverse(String s) {
    Deque<Character> st = new ArrayDeque<>();
    // TODO 9: push every character of s
    StringBuilder sb = new StringBuilder();
    // TODO 10: pop until empty, append to sb
    return sb.toString();
}
// reverse("STACK") gives "KCATS"
```

Your task

Use Java's built-in Deque as a stack

Push all letters, then pop all

Test: STACK, level, empty string

Task 7: balanced brackets

```
public static boolean balanced(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        // TODO 11: opener -> push
        // TODO 12: closer -> pop and compare;
        //   empty stack or mismatch -> false
    }
    return st.isEmpty();
}

static boolean matches(char o, char c) {
    return (o == '(' && c == ')')
        || (o == '[' && c == ']')
        || (o == '{' && c == '}');
}
```

Test these

(a+b)*c gives true

[(a+b)*c] gives true

(a+b] gives false

((a+b) gives false

a+b) gives false

Extension challenges

Postfix

Evaluate

Evaluate 5 1 2 + 4 * + 3 - with a stack of ints. Expected result: 14.

Browser

History

Model Back and Forward with two stacks.

Size

Count

Add size() to LinkedStack in O(1) using a counter.

Submission checklist

Item	What to check
Files	Node, SinglyLinkedList, DoublyLinkedList, LinkedStack, Main
Output	Screenshot of Checkpoint 1 and Task 5 output
Code quality	Clear names and a comment for each pointer change
Testing	Tried empty, one-node and many-node cases

Submit as instructed on your course platform.

Lab recap and reflection

A linked list is a Node plus head; operations are pointer rewiring

Doubly linked lists add prev and tail for work at both ends

A stack is a list restricted to one end: push and pop at the head

One pass with a stack solves reversing and bracket matching

Always test: empty, one node, last node

Next: queues (FIFO), the partner of stacks